MAANG TECHNICAL INTERVIEW PREPARATION

JavaScript Engine Architecture & Internals

Module 1: Code Parsing, Tokenization & AST

Q1. What happens between writing JavaScript source code and executing it?

The journey from high-level JavaScript source code to physical execution on the CPU follows a structured pipeline:

1. **Lexical Analysis (Scanner):** Converts the raw code character stream into atomic tokens (e.g., `const`, `x`, `=`, `10`).
2. **Syntactic Analysis (Parser):** Validates syntax rules and builds an **Abstract Syntax Tree (AST)**.
3. **Bytecode Generation:** An interpreter (e.g., V8's Ignition) converts the AST into portable bytecode.
4. **JIT Compilation:** High-frequency hot functions are compiled into optimized machine assembly by a JIT compiler (e.g., TurboFan).

FIGURE 1: JAVASCRIPT ENGINE EXECUTION PIPELINE



Q2. What is the difference between a parser and an AST?

A **parser** is an active processing algorithm that reads tokens and checks grammatical correctness. An **AST (Abstract Syntax Tree)** is the static tree data structure produced by the parser representing the program logic without formatting details.

Q3. Why might a JS engine use bytecode instead of compiling directly to machine code?

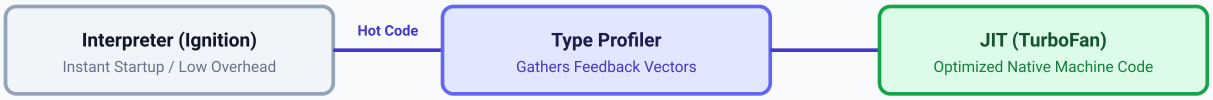
Direct-to-machine-code compilation produces large binary sizes that consume excessive memory on client devices. **Bytecode** is compact (~5x smaller than machine code), enables instant execution start times, and is arch-independent across x86, ARM, and MIPS chips.

Module 2: JIT Compilation & Speculative Optimization

Q4. How do an interpreter and a JIT compiler differ?

An **interpreter** executes code line-by-line immediately without initial compile delays, but runs loops slowly. A **Just-In-Time (JIT) compiler** monitors runtime execution, identifies hot functions, and compiles them directly into high-speed native CPU assembly code.

FIGURE 2: ADAPTIVE EXECUTION & TIERING ARCHITECTURE



Q5. Why can JIT compilation improve performance but increase startup costs?

JIT compilers achieve high runtime throughput by compiling bytecode to hardware machine instructions. However, compiling code consumes initial CPU cycles and memory for type profiling and IR generation, increasing initial boot latency.

Q6. What is speculative optimization, and why is it useful in a dynamically typed language?

JavaScript lacks static typing. Without compile-time type guarantees, native assembly would require expensive runtime type checks before every operation.

> **Speculative Assumption:** The JIT compiler observes runtime type patterns (e.g., `add(a, b)` has always received integers) and speculatively compiles machine code assuming types will never change, omitting costly runtime checks.

Module 3: Deoptimization & Inline Caches

Q7. What causes deoptimization, and does every type change trigger it?

Deoptimization occurs when speculative assumptions made during JIT compilation are violated (e.g., passing a string into a function optimized for integers). The engine discards optimized machine code and returns to interpreter bytecode.

Does every type change trigger it? No. Polymorphic call handling allows JIT compilers to handle multiple expected types without triggering deoptimization.

FIGURE 3: SPECULATIVE EXECUTION & DEOPTIMIZATION FLOW



Q8. How do inline caches (ICs) accelerate property access?

An **Inline Cache (IC)** avoids dynamic dictionary lookups by recording the structural memory offsets of object properties directly at property access call sites. When an object with an expected shape is encountered, the IC fetches property values via instant direct memory offset math.

Q9. What are hidden classes in V8, and why do object property layouts matter?

JavaScript objects are specification-wise dynamic key-value maps. To avoid slow hash map searches, V8 assigns hidden, immutable internal descriptors called **Hidden Classes (or Shapes)**. Shapes map property names to fixed memory offsets inside object structures, making property accesses as fast as compiled C++ struct accesses.

Module 4: Shapes, Transitions & Access Polymorphism

Q10. How do shapes and hidden classes relate to one another?

Shapes and **Hidden Classes** are identical concepts across different engine implementations (V8 calls them *Maps*, SpiderMonkey calls them *Shapes*, JavaScriptCore calls them *Structures*). They store key-to-offset metadata maps and layout transition pointers.

FIGURE 4: HIDDEN CLASS TRANSITION CHAIN



Q11. Can two different objects share the same hidden class?

Yes. If two distinct objects are instantiated with the exact same property assignment sequence (e.g., initialized via the same constructor or class), they point to the exact same Shape in memory, maximizing engine caching efficiency.

Q12. What is the difference between monomorphic, polymorphic, and megamorphic property access?

- **Monomorphic (1 Shape):** The call site sees exactly 1 shape. Highly optimized fast-path execution.
- **Polymorphic (2–4 Shapes):** The call site encounters a small set of distinct shapes. Handled via a small inline lookup table.
- **Megamorphic (>4 Shapes):** The site encounters numerous unpredictable shapes. The IC falls back to slow global hash map lookups.

Module 5: Property Layouts, Inlining & Bailouts

Q13. Why can objects created with different property orders have different shapes?

Shape transitions form a deterministic tree graph. Initializing an object as `{x:1, y:2}` follows transition chain `C0 → C_x → C_xy`. Initializing as `{y:2, x:1}` follows transition chain `C0 → C_y → C_yx`. Because offset orders differ, `C_xy` and `C_yx` are distinct Shapes.

FIGURE 5: FUNCTION INLINING TRANSFORMATION



Q14. How can an optimizing compiler inline a function?

Function inlining replaces a function invocation directly with the callee function's body code. This eliminates stack frame push/pop overhead, pointer tracking, and parameter setup expenses, opening opportunities for further optimizations like dead code elimination.

Q15. What is a bailout, and how does it relate to deoptimization?

A **bailout** is the runtime control transfer executed when a deoptimization check fails. It restores interpreter registers, reconstructs the call stack frame from native machine code, and seamlessly resumes bytecode execution.

Module 6: Garbage Collection & Memory Leaks

Q16. How does a tracing garbage collector determine which objects can be reclaimed?

Modern JS engines use **Mark-and-Sweep tracing garbage collection** based on *Reachability*. Traversal originates from **GC Roots** (Global objects, Active Call Stack variables, DOM tree references). Objects unreachable at the end of traversal are marked as garbage and reclaimed.

FIGURE 6: MARK-AND-SWEEP REACHABILITY GRAPH



Q17. Why do circular references not necessarily cause memory leaks?

Legacy engines used primitive **Reference Counting**, which leaked memory when objects referenced each other cyclically. Modern **Mark-and-Sweep** collectors check reachability from GC Roots: if a cyclic pair is disconnected from GC Roots, both are swept regardless of their mutual references.

Q18. How can closures and global caches contribute to memory leaks?

Closures retain persistent references to their enclosing scope variables. If a closure remains attached to a long-lived object (e.g., an event listener), all outer variables remain pinned in memory. Similarly, unbounded global cache objects continuously retain references without eviction.

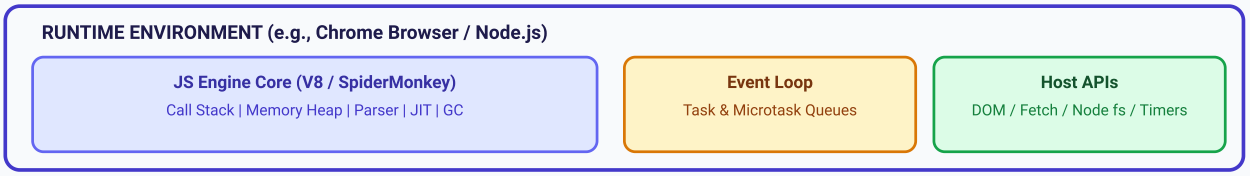
Module 7: Advanced GC Strategies & Runtime Architecture

Q19. What are generational, incremental, and concurrent garbage collection?

To avoid frame drops and application freezes, modern engines utilize three advanced GC strategies:

- **Generational GC:** Based on the *Weak Generational Hypothesis* (most objects die young). Segregates memory into **New Space** (fast, frequent Scavenge collections) and **Old Space** (infrequent full collections).
- **Incremental GC:** Splits major GC marking passes into tiny time slices interleaved between UI rendering frames to prevent frame stuttering.
- **Concurrent GC:** Runs marking and sweeping tasks on background worker threads simultaneously while main application JS continues executing.

FIGURE 7: JS ENGINE VS RUNTIME HOST ARCHITECTURE



Q20. What is the difference between the JavaScript engine and the runtime environment?

The **JavaScript Engine** (e.g., V8, JavaScriptCore) is the core execution component that parses ECMAScript code, manages memory, generates bytecode/machine code, and runs garbage collection.

The **Runtime Environment** (e.g., Chrome Browser, Node.js) embeds the JS Engine and provides platform-specific infrastructure, including the Event Loop, Async Task Queues, and host Web/Node APIs (DOM, Fetch, File System).